High Performance Parallel Computing
Homework 2

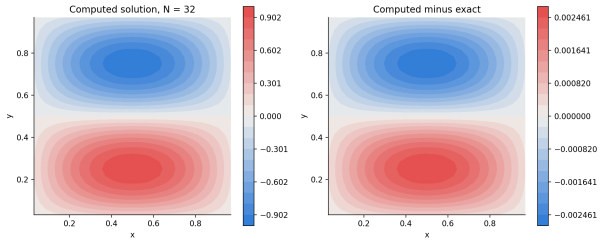
Liam Pohlmann, Alex Shaffer

October 12, 2026

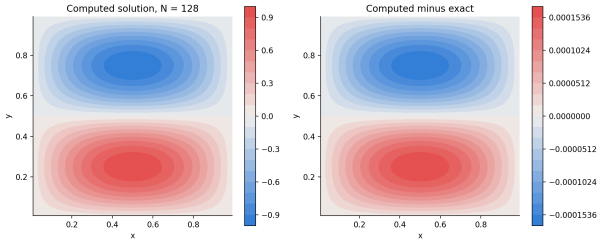
1 Problem 1

1.1 Verification

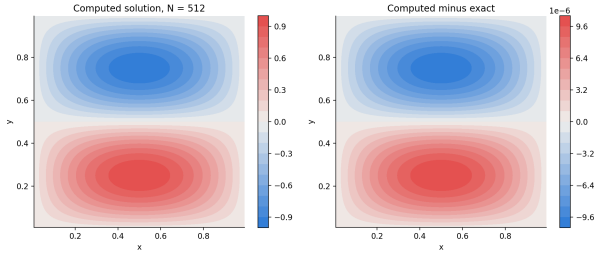
As demonstrated with Figures 1 and 2, the implemeted algorithm converges to the correct solution with the expected quadratic convergence rate.



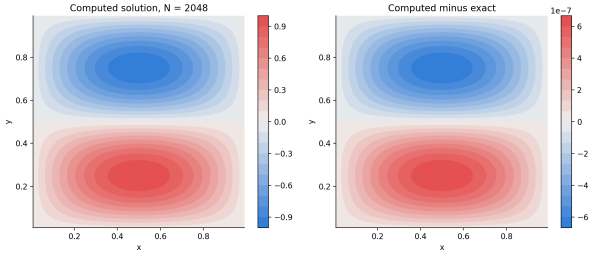
(a) $N = 32$



(b) $N = 128$



(c) $N = 512$



(d) $N = 2048$

Figure 1: Computed solution (left panel of each pair) and the difference between the computed and exact solutions (right panel) for each grid size $N = N_x = N_y$.

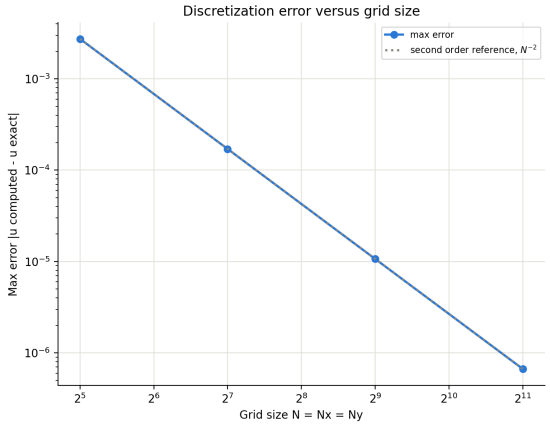
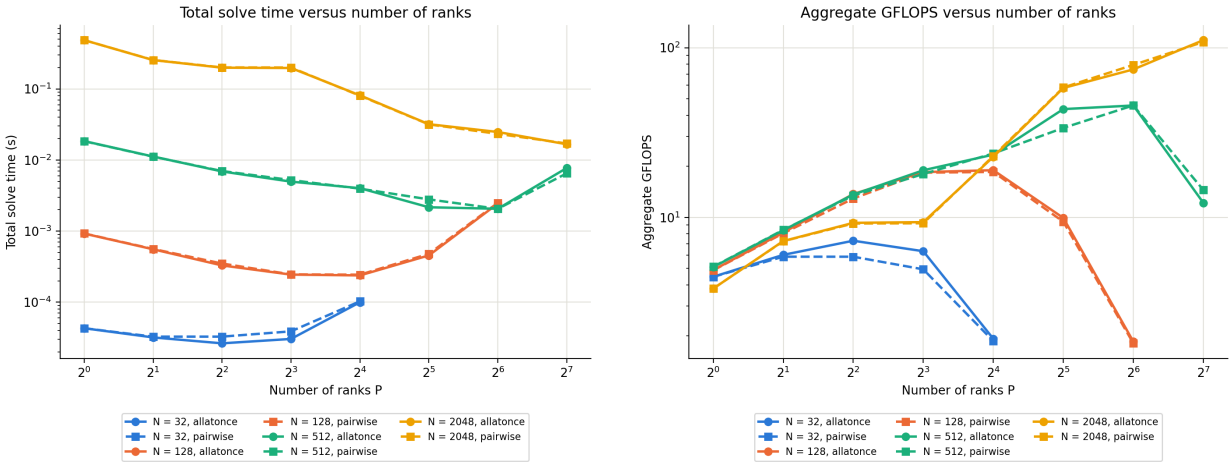


Figure 2: Maximum error of the computed solution versus grid size $N = N_x = N_y$, with an N^{-2} reference line.

1.2 Parallel Scalability

As indicated by Figure 3, the benefits of increased number of ranks in a simulation is very much a function of the size of the problem (N, in this case). For example, as indicated by Figure 3a, the maximum compute time for a single problem size is *maximized* when $P = N/2$, which is in some ways counterintuitive, as one would normally expect the compute time to *strictly decrease* with the number of processors. Interestingly, while the gains in GFLOPS achieved by the increase in processors is clear, the decrease in compute time as a function of processors is marginal at best, given that the total solve time goes from less than a second to less than less than a second[1]. The above results would indicate that gains from parallelism would perhaps be better seen by increasing the problem size further...



(a) Total solve time

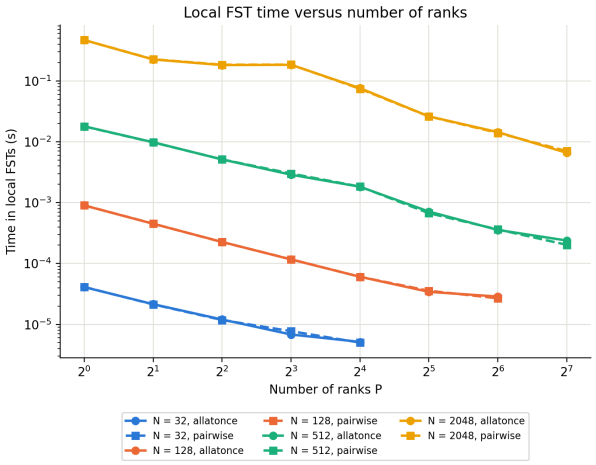
(b) Aggregate GFLOPS

Figure 3: Total solve time and aggregate GFLOPS versus the number of ranks P for each grid size N and both transpose strategies (solid: all-at-once, dashed: pairwise).

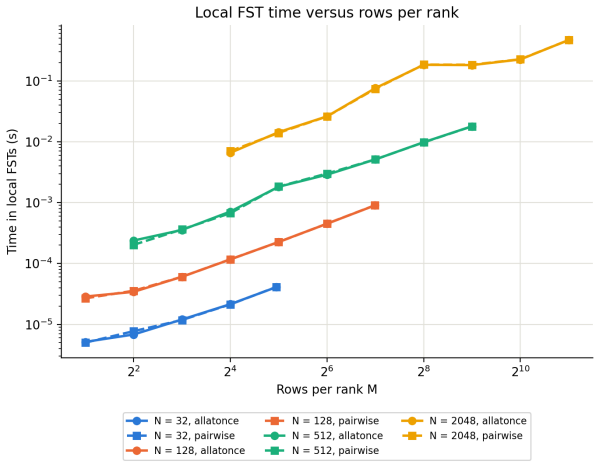
[1]This is indeed stated with a bit of tongue in cheek.

1.3 Transpose Comparison

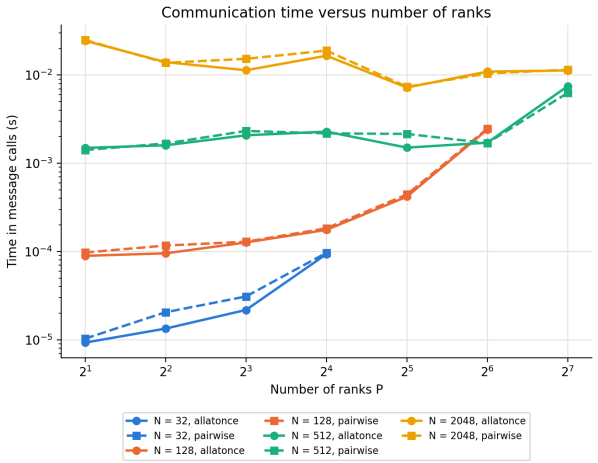
In Figure 4 we compare the communication of the two transpose methods across four different values of n. In the plot Figure 4a shows a largely negligible difference in the two algorithms. The differences in there performances there vary far less than how each method scales with ranks. Figure 4b tells a similar story. The plot which shows to a greater degree how these two differ is Figure 4c. Here we see the all-at-once largely outperform the the pairwise case. Notably, the gap is largest at small n where the latency dominates. It narrows at larger n where this is not the case.



(a) Local FST time versus P



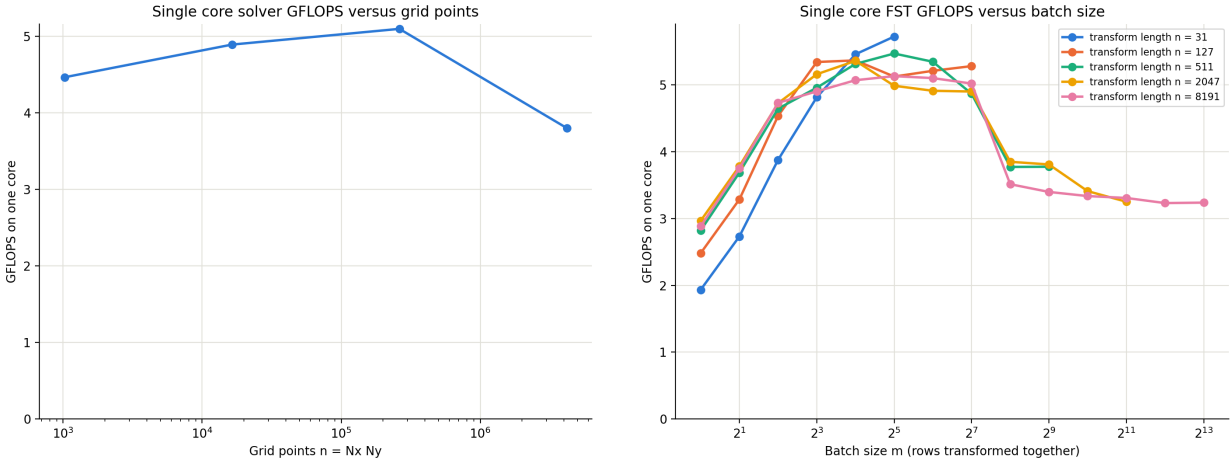
(b) Local FST time versus M



(c) Communication time versus P

Figure 4: Time spent in the local fast sine transforms, versus the number of ranks P and versus the rows per rank M, and time spent in message calls versus P, for each grid size N and both transpose strategies.

1.4 Single Core



(a) Full solve versus grid points $n = N_x N_y$

(b) Batched FST versus batch size m

Figure 5: Single-core GFLOPS of the full Poisson solve and of the batched fast sine transform alone, the latter for each transform length n.

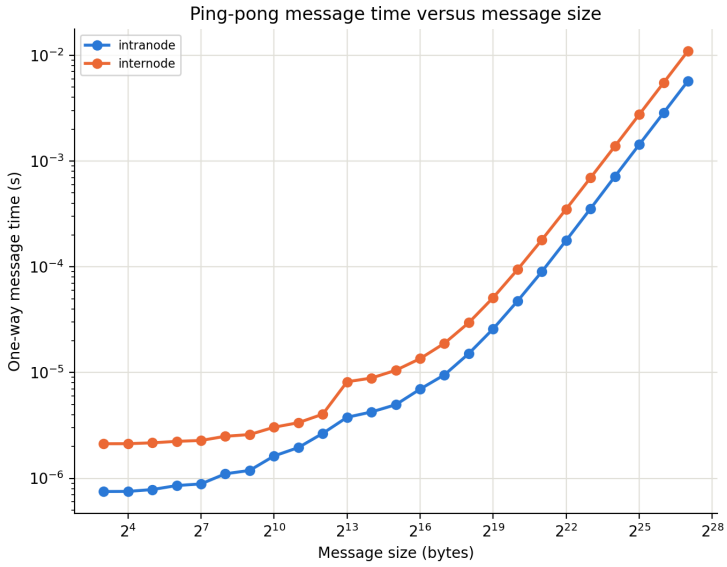


Figure 6: One-way point-to-point message time versus message size from the ping-pong test, with both ranks on one node (intranode) and on two different nodes (internode).

In general, once parallel efficiency is maximized, continued gains are limited by the communication overhead. This is because the computation time per node starts to rival the communication time, so while an increase in the number of nodes might continue to decrease the computation time, the communication needs continue to grow.

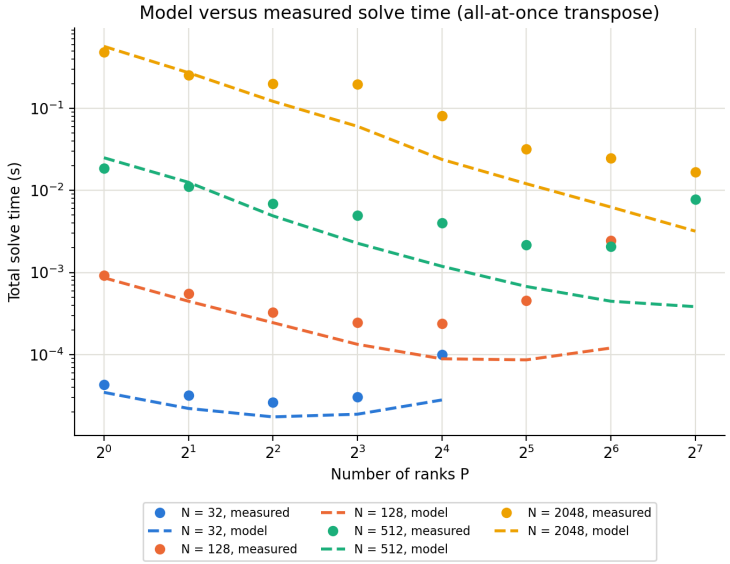


Figure 7: Measured total solve time (markers) and the performance model (dashed lines) versus the number of ranks P for each grid size N, all-at-once transpose.

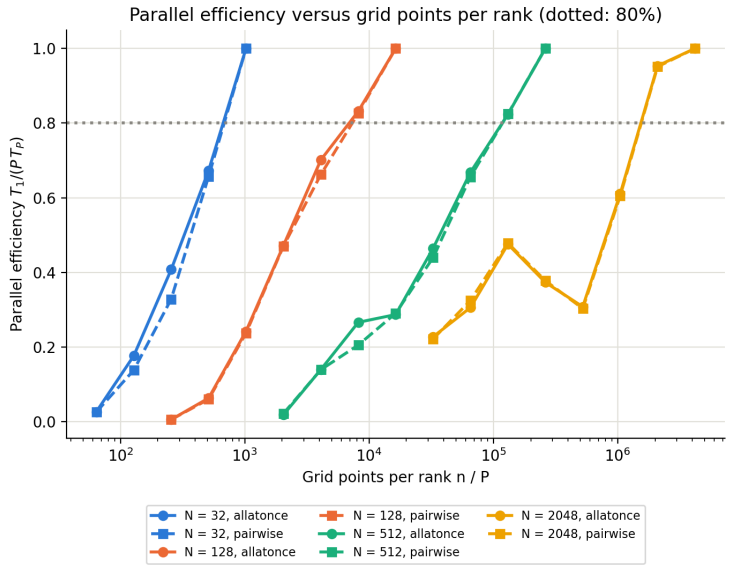


Figure 8: Parallel efficiency $T_1/(P\,T_P)$ versus grid points per rank n/P for each grid size N and both transpose strategies. The dotted line marks 80% efficiency.